

Ship Contracts in a TS

Abstract

This paper suggests that Contracts should ship in a Technical Specification first, not in the C++26 IS.

The motivation for this is many-fold. We lack

- implementation experience
- more importantly, field experience in the form of deployment and usage experience
- WG21-wide design consensus on multiple points in P2900
- cohesion and coverage; we don't have WG21-wide consensus on how to deal with virtual functions, how to deal with coroutines, and how to deal with pointers to functions
- increased safety; contract assertions are subject to as much undefined behavior as the rest of the language is, and this may well be very problematic both for the runtime checks written in contract assertions and also for the chances of static analysis tools' ability to correctness-check code with the help of contract assertions.

Additional ruminations

There isn't actually much to add.

For those unaware, we have had language TSes before, for Concepts, Modules, and Coroutines. Those were, regardless of the grumbling about the time and effort they took, all successful in the sense that we got better language features due to using a TS, with fewer if any unaddressed concerns, with a better understanding, and better consensus in WG21.

We have various design disagreements in SG21, and based on EWG's look at Contracts in Tokyo, fair amounts of disagreements between what SG21 is currently proposing and what some portions of EWG think about it. We can certainly try to resolve all such differences and ship the result in C++26, but that seems unwise. We should hopefully have more than paper-exercise disagreements that are resolved on paper - we should try to resolve those disagreements with something more concrete than papers in our hands, and that something more concrete should be implementations we can play with.

And that includes implementations of different alternatives instead of just one, to have an antithesis for the thesis, with the goal of producing a better synthesis.

Finally, there's been various discussions in SG21, including significant parts of the discussion on [P2680](#) where statements along the lines of "if we want to ship this in C++26, we should do \$foo and \$bar, but not \$baz" have been made. That is, the schedule goal for shipping the Contracts MVP has a significant impact on what we look at and when. While that's understandable for an MVP approach, it also runs into the danger of closing alternative design doors prematurely, when they don't fit into the MVP schedule and the timeline to make it into C++26 - even when they might be better doors to walk through.

So, the main goal of this idea, in addition to getting a better understanding, more field experience, and all that jazz, is to remove that urgency, and hopefully remove its significant impact on the design choices that we make. Decouple Contracts from such an immediate deadline, have a breather, gather more experience. Use the ISO trial balloon that is a Technical Specification to do it.

What questions should the TS answer?

Here's a start:

- is it ergonomic:
 - does constification cause issues with teaching? When reading? Does it catch more bugs than it causes? How cumbersome is it?
 - is it useful for its intended purpose or does it end up being used for something else?
 - can compilers provide a user-friendly interface to the implementation-defined features that users can integrate with their toolchains and actively use as intended by the design?
 - can users understand multiple evaluation?
- is it fully-featured?
 - are there immediate missing pieces one reaches for but they don't exist? for precedent, see lambdas in unevaluated contexts.
 - how badly do we need Lisa's full interfaces as opposed to just post-conditions with captures sugar?
 - how badly do we need labels?
- is it implementable?
 - the core proposal is, but the linking callee/caller parts are, while clearly doable in a strict sense of the word, a question when it comes to linker support. Deployment experience with this will be useful.
 - is the evaluation elision between postconditions/preconditions implementable in a useful way? (We might not get this from a compiler yet, it requires work, but I don't want to standardize something like ``throw;`` that prevents efficient exception implementations without thread-local storage. It might be we find that we have to allow more things than just side-effects to be ignored, like `longjump`.)

I'll add a couple more:

- Does it provide a base that we can build useful static analysis on?
- If we were to put both P2900 and P3285 (Gaby's proposal defaulting to 'strict' contracts, providing 'relaxed' contracts as an alternative) into the same TS, which approach is more viable and more useful?